stviz-animate Technical Documentation

Last updated: 2026-11-13

Table of contents

- Overview
- Repository layout
- Build and runtime
- Data format (.stviz)
- Data conversion pipeline
- Rendering architecture
- UI architecture and state
- Timeline system
- Advanced timeline
- Color modes and filtering
- Sample grid system
- Export pipeline
- Output and logs
- Packaging and distribution
- Configuration and environment
- Troubleshooting
- Extension points

Overview

stviz-animate is a desktop viewer and animator for spatial transcriptomics datasets. It loads `.stviz` files, renders point clouds on the GPU, and provides a timeline for animating transitions between embedding spaces. It supports categorical, continuous, and gene-based coloring, sample grid visualization, and both screenshot and loop video export.

Key runtime goals:

- Fast GPU point rendering for large datasets.
- Predictable timeline playback and export.
- Lightweight on-disk format with memory mapping.

Repository layout

- `src/` Rust application code.
  - `app/`: the egui application, split by concern. `mod.rs` holds the `StvizApp` state, setup and the per-frame `update` loop; every other file adds an `impl StvizApp` block for one area:
    - `left_panel.rs`, `timeline.rs`, `advanced.rs`, `viewport.rs`: the four UI regions (settings panel, bottom timeline, advanced node editor, central view).

        - `dataset.rs`, `keyframes.rs`, `layout.rs`: loading data, color/filter recomputation, keyframe bookkeeping, bounding boxes and the sample grid.
        - `export.rs`, `convert.rs`, `env.rs`: video/screenshot export, `.h5ad` conversion and mock data, and locating Python/ffmpeg/app directories.
        - `export_job.rs`: the background worker for slow export steps (ffmpeg/OpenCV encoding, opt-in ffmpeg install). It reports log lines over a channel; `poll_export_job` applies them each frame.
        - `workspace.rs`, `input.rs`, `render_callback.rs`, `util.rs`: workspace save/restore, hotkeys and playback ticking, the wgpu paint callback, and small shared helpers.
        - `settings.rs`, `process.rs`: preferences persisted between launches, and cancellable subprocess helpers (captured or line-streamed output) used by the conversion and export workers.
        - `style.rs`, `window.rs`: categorical palettes and theme visuals; fullscreen, display refresh rate and WSL detection.
    - `render.rs`: WGPU pipeline and point cloud rendering.
    - `data.rs`: `.stviz` loader, memory mapping, metadata parsing.
    - `camera.rs`: 2D camera controls (pan, zoom, fit bbox).
    - `color.rs`: color utilities and palettes.
    - `main.rs`: app entrypoint, icon loading.
- `python/` conversion and export helpers.
    - `export_stviz.py`: `.h5ad` to `.stviz` exporter.
    - `mock_spatial_dataset.py`: mock dataset generator.
    - `export_video_cv2.py`: OpenCV fallback for video export.
- `assets/`: shaders, icons, and static resources.
- `scripts/`: packaging scripts for Windows, macOS, and Ubuntu.
- `docs/`: technical documentation (this file).
- Runtime output is stored in an OS user-data directory:
    - Windows: `%APPDATA%/stviz-animate/`
    - macOS: `~/Library/Application Support/stviz-animate/`
    - Linux: `${XDG_DATA_HOME:-~/.local/share}/stviz-animate/`

Build and runtime

Build and run (debug):

```
cargo run
```

Release build:

```
cargo run --release
```

Dependencies:

- Rust toolchain.
- GPU drivers for accelerated rendering.

- Python 3.9+ for conversion and optional OpenCV fallback export.
- Converter dependencies are specified in `python/requirements.txt` with versions chosen for broad wheel availability.

Data format (.stviz)

`.stviz` is a custom binary format designed for fast memory mapping and low overhead parsing.

File layout:

- Bytes 0..7: magic bytes `STVIZ\0\0\0`.
- Bytes 8..11: little-endian `u32` format version.
- Bytes 12..19: little-endian `u64` JSON metadata length.
- Bytes 20..(20+json_len-1): JSON metadata (`StvizMeta`).
- Padding to 16-byte alignment.
- Binary data blocks (spaces, obs, expression arrays) referenced by offsets in metadata.

Metadata (`StvizMeta`):

- `version`: format version.
- `n_points`: number of cells/points.
- `spaces`: list of 2D spaces with offsets and bounding boxes.
- `obs`: list of categorical/continuous annotation columns.
- `expr`: optional CSC gene expression metadata.

Space metadata (`SpaceMeta`):

- `name`: space identifier (e.g., `spatial`, `X_umap`).
- `dims`: dimensionality (currently 2D).
- `offset` + `len_bytes`: byte range for packed `f32` positions.
- `bbox`: `[min_x, min_y, max_x, max_y]`.

Obs metadata (`ObsMeta`):

- Categorical: `u32` labels array + category names + optional RGBA palette.
- Continuous: `f32` values array.

Expression metadata (`ExprMeta`):

- Compressed Sparse Column (CSC) matrices.
- `indptr`, `indices`, `data` arrays and `var_names` for genes.

`data.rs` maps the file into memory, validates block bounds/lengths/consistency, and provides zero-copy slices. Unsupported format versions are rejected.

Data conversion pipeline

Design goal: users drop a `.h5ad` and it works. The managed Python environment, dependency install and progress details are kept out of the way: the UI shows short phases ("Setting up (first time

only)...", "Converting... 40%"), and everything else goes to the collapsed Conversion log and `<user-data>/logs/`.

User-data folder (converted files, logs, settings, the managed venv):

- Windows: `%APPDATA%/stviz-animate/`
- macOS: `~/Library/Application Support/stviz-animate/`
- Linux: `${XDG_DATA_HOME:-~/.local/share}/stviz-animate/`

Manual conversion:

```
python -m pip install -r python/requirements.txt
python python/export_stviz.py --input your_data.h5ad --output your_data.stviz --include-expr
```

Optional flags: `--layer NAME` or `--use-raw` to choose the expression matrix, and `--gene-symbols COLUMN` to take gene names from a `var` column.

`.h5ad` to `.stviz` (`python/export_stviz.py`)

- Opens the file with `h5py` and reads only `obs`, the first two columns of each `.obsm` entry, the `*_colors` palettes and gene names (via `anndata`'s element readers). The full `AnnData` is never loaded.
- Spaces: every `.obsm` entry with at least two numeric columns (e.g., `X_umap`, `X_pca`, `spatial`); the first two dimensions are used. Non-finite coordinates are stored as NaN (not drawn) and left out of the bbox.
- Falls back to `obs` columns (`x/y`, `spatial_x/spatial_y`, etc.) if needed.
- Categorical obs: categorical, string and boolean columns (categories + labels + optional scanpy palette; `#RGB`, `#RRGGBB(AA)`, common names and RGB(A) tuples are understood). Columns that look like per-cell ids (more than half the cells distinct) are skipped.
- Continuous obs: numeric columns (float values, NaN for missing). Integer columns with at most 50 distinct values are exported as categorical as well.
- Optionally exports gene expression as CSC for fast gene lookup. The source matrix (`X`, `--use-raw`, or `--layer`; CSR, CSC or dense on disk) is streamed in chunks in two passes (count per gene, then fill) directly into the output file. More than `u32::MAX` non-zeros or cells is reported as an error.
- The output is written to `<output>.partial` and renamed into place, so an existing `.stviz` (possibly memory-mapped by the app) is never modified in place.
- The metadata records `source` (the input path) so the app can tell which `.h5ad` a file came from.

In the app (`src/app/convert.rs`) conversion runs as a `ConvertJob` on a worker thread: it creates the managed venv and installs dependencies only when missing, streams script output into the conversion log, and can be cancelled (the subprocess is killed and partial output removed). `.stviz` files are opened and validated on a worker thread too (`start_dataset_load` in `src/app/dataset.rs`).

Tests: `python python/tests/test_export_stviz.py` (round trips for CSR/CSC/dense, layers, raw, column typing and error handling), and `cargo test -- --ignored` for an end-to-end mock generation + conversion (needs Python and internet).

Mock dataset generation (`python/mock_spatial_dataset.py`)

- Generates synthetic spatial and embedding spaces.
- Adds multiple categorical and continuous features.
- Used to test workflows and GPU performance.

Video export fallback (`python/export_video_cv2.py`)

- OpenCV-based MP4 encoding when ffmpeg is not available.
- On first fallback usage, OpenCV is installed into the managed converter venv (internet required).

Rendering architecture

Rendering is GPU-first using WGPU and a point-sprite shader.

Key components:

- `PointCloudGpu` manages GPU buffers, bind groups, and pipelines.
- `Uniforms` contain camera transforms, interpolations, and point size.
- `assets/pointcloud.wgsl` renders instanced quads with a circular mask.

Uniforms (core fields):

- `viewport_px`: viewport size in pixels.
- `center`, `pixels_per_unit`: camera transform.
- `t`: timeline interpolation factor.
- `point_radius_px`: point size in pixels.
- `from_center/from_scale`, `to_center/to_scale`: per-space transforms.
- `mask_mode`: toggles round vs square points.

Offscreen render (screenshots and high-quality export):

- Uses MSAA (4x) for anti-aliasing of points.
- Renders into a multisampled texture and resolves to a single-sample texture for PNG output.

UI architecture and state

`StvizApp` holds all UI state:

- Dataset state (loaded dataset, path, ids).
- Timeline state (keyframes, times, ease, playback).
- Rendering state (camera, point size, shaders, buffers).
- Export state (fps, duration, quality, logs, cancellation).
- Sample grid state and filters.
- Advanced timeline state (cards, connections, selection).

UI layout:

- Left panel: conversion, dataset controls, color/filter controls, export controls.
- Central panel: viewport rendering.
- Bottom panel: timeline and playback controls.
- Separate viewport: advanced timeline canvas.

Viewport interactions:

- Pan by dragging.
- Zoom by mouse wheel.
- Reset scaling via timeline controls.

Timeline system

The timeline controls transitions between spaces and colors.

Core structures:

- `space_path`: list of space indices (keyframes).
- `color_path`: list of color keys (categorical/continuous/gene).
- `key_times`: normalized time positions for each keyframe.
- `ease_mode`: linear, smoothstep, sine, or quad.

Playback:

- `speed`: units per second.
- `playback_mode`: Once, Loop, PingPong.
- Keyboard controls for play/pause and frame stepping.

Advanced timeline

The advanced timeline is a separate viewport window with a canvas UI.

Features:

- Card-based timeline with draggable cards.
- Grid mode (snap cards to fixed grid size).
- Card inspector panel with space and color selection.
- Per-card duration controls.
- Multi-selection via marquee selection.
- Connections between cards (inputs/outputs).
- Preview mode (isolates the selected card when not playing).

Cards can include optional filters to display only subsets of cells for that segment.

Color modes and filtering

Color modes:

- Categorical: per-category palette with optional overrides.

- Continuous: gradient palette based on numeric values.
- Gene: gene expression coloring (if `expr` is present).
- "Clip outliers (1-99%)" maps continuous and gene colors onto the 1st-99th percentile range (for genes, the 99th percentile of expressing cells) instead of the full min/max.

Filtering:

- Filters are applied immediately when categories are toggled.
- If all categories are disabled, nothing is drawn (empty draw list).
- Filters are reflected in draw indices, downsampling, and sample grid layout.

Sample grid system

The sample grid arranges spatial samples into a regular grid:

- Enabled in the left panel.
- Uses a categorical field (default `sample`) to group points.
- Optionally respects filters.

Grid layout:

- Computes per-category bounding boxes in the selected space.
- Places categories into a grid with adjustable padding.
- Supports labels, including custom label overrides.

Labels:

- Optional overlay showing group labels above each grid tile.
- Uses contrasting color relative to the background.

Export pipeline

Screenshots

- Always rendered at 4K (3840×2160).
- Uses offscreen GPU rendering with MSAA.
- Output path: `<user-data>/output/screenshots/stviz-animate_screenshot_*.png`.
- PNG encoding is lossless.

ffmpeg and the fallback encoder

- ffmpeg is used when found (see `resolve_ffmpeg_path`). Otherwise frames are encoded with OpenCV (`python/export_video_cv2.py`), installed into the managed venv on first use, so export always works.
- The Output panel keeps the opt-in ffmpeg install (winget/choco, brew, apt/dnf/pacman/zypper) under a collapsed **Faster video encoding** header, shown only when ffmpeg is missing. It is off by default because it runs the system package manager, which may need admin rights.

Loop video export

Pipeline:

1. Render PNG frames into `<user-data>/output/exports/loop_<timestamp>/`.
2. Encode into a temporary MP4 in the export directory.
3. Move/rename to final output in `<user-data>/output/` only on success.
4. Optionally delete frames after encoding.

Quality options:

- Render resolution: Current viewport, Full HD, 4K.
- Encoding presets:
  - Standard: CRF 23, preset medium, yuv420p.
  - High: CRF 18, preset slow, yuv420p.
  - Ultra: CRF 14, preset slow, yuv420p.

Export controls:

- Cancel export stops rendering and deletes only managed temporary output.
- Export log is stored in `<user-data>/logs/export_log_*.txt` and visible in the UI.

Output and logs

- Conversion outputs: `<user-data>/output/*.stviz`.
- Mock datasets: `<user-data>/output/mock_spatial_*.h5ad` (temporary).
- Videos: `<user-data>/output/*.mp4`.
- Screenshots: `<user-data>/output/screenshots/stviz-animate_screenshot_*.png`.
- Logs: `<user-data>/logs/convert_log_*.txt`, `<user-data>/logs/export_log_*.txt`.

Packaging and distribution

Releases are built by `.github/workflows/release.yml`. Pushing a tag such as `v1.1.0` builds all three bundles with the scripts below and attaches them to a GitHub Release. The tag must match the `version` in `Cargo.toml`. Running the workflow by hand from the Actions tab builds the bundles as downloadable artifacts without publishing. Bundles are written to `dist/`, which is not tracked in git.

Scripts:

- Windows: `scripts/package_windows.ps1`
- macOS: `scripts/package_mac.sh`
- Ubuntu: `scripts/package_ubuntu.sh`

Bundle notes:

- macOS bundle includes `python/` under `Contents/Resources`.
- Linux packages expect `python/` under `/usr/share/stviz-animate/python`.
- Windows bundle can ship `ffmpeg.exe` alongside the executable.

Configuration and environment

Python selection (`detect_python_cmd` in `src/app/env.rs`):

- Prefers an active venv/conda (`VIRTUAL_ENV`, `CONDA_PREFIX`), then `python3`, `python` (Linux/macOS) or `python`, `py`, `python3` (Windows; `py` is the python.org launcher, present even when Python is not on PATH).
- Interpreters older than 3.9 are rejected with a message.
- On Windows the Microsoft Store Python (`...\WindowsApps\...`) is only used if nothing else is installed: it redirects writes under AppData to a private folder, so the venv it creates is invisible to the app. If that happens the conversion stops with a message pointing to python.org.
- Managed converter env location: `<user-data>/.stviz_venv` (auto-created and dependency-installed on first conversion/export-fallback run).
- Every Python subprocess gets `PYTHONUTF8=1` and `PYTHONIOENCODING=utf-8`, so paths with non-ASCII characters print correctly on Windows (whose pipes otherwise use the legacy code page).

Environment variables:

- `STVIZ_FFMPEG`: override ffmpeg path.
- `STVIZ_DATA_DIR`: override the user-data directory root.

GPU selection:

- Uses WGPU default adapter.
- CPU adapters enable fast render mode by default.

Persistent settings:

- `<user-data>/settings.json` stores appearance, a few defaults (include expression, export fps/duration, outlier clipping, ffmpeg auto-install), the last used folder and recent datasets. It is written shortly after a change and on exit; an unreadable file falls back to defaults.

Command line:

- `stviz-animate <file.h5ad|file.stviz>` opens the file at startup.

First-run help (`src/app/help.rs`):

- `ui_welcome` replaces the viewport while no dataset is loaded: open a file, generate demo data, recent datasets, a short guide, and conversion progress with Cancel.
- `ui_top_bar` holds the app version, **Open data…** and **Shortcuts (?)**. `ui_shortcuts_window` lists `SHORTCUTS` and is toggled with `?` / `F1` in `handle_hotkeys`; Esc closes it first.

Labels overlay (`src/app/overlay.rs`):

- `OverlaySettings` (title, legend, space name, scale bar, units per coordinate, label size) is saved in workspaces.
- `overlay_content` decides what to show for the current time: the legend follows the colour key of the segment half the view is in, and the scale bar only appears while the view rests on a space whose name contains "spatial" and the sample grid is off.

- `layout` turns that into primitives sized relative to the output (1 unit = 1 px at 1080p). The viewport draws them with egui as a preview; exports rasterize them on the CPU (`Canvas`, using the bundled Ubuntu-Light font through `ab_glyph`) into each frame after GPU readback, for both direct ffmpeg streaming and PNG frames.

Workspaces (`src/app/workspace.rs`):

- JSON, `version: 1`; fields added later are optional so older files still open.
- Stores keyframes, colors, filters, view, sample grid, export settings and the advanced timeline layout (card positions, sizes, durations, connections).
- The dataset is looked up at the saved path, then next to the workspace file by file name.
- Opening validates the dataset and every referenced space/field before changing the current session.

Subprocesses (`src/app/process.rs`):

- `run_cancellable` (captured output) and `run_streaming` (line by line) poll the child and stop it when the job's cancel flag is set.
- Cancel kills the whole process tree (`kill_tree`): on Linux/macOS each child runs in its own process group, which is killed as a group; on Windows `taskkill /T /F` is used. This matters on Windows, where a venv's `python.exe` is a launcher that runs the real interpreter as a child, and for pip, which starts its own children. After a cancel the output pipes are not waited on, so a surviving process can't hang the app.
- All subprocesses are started with `CREATE_NO_WINDOW` on Windows, so no console windows flash up.

Windows notes:

- File locking: a loaded `.stviz` is memory-mapped, and Windows cannot replace or delete a mapped file. Before a conversion overwrites the loaded dataset, or a previous mock dataset is deleted, the app closes it (`release_dataset_file`); on exit the dataset is released before mock files are cleaned up. If the file is open in another stviz-animate window, the converter reports that instead of failing with a raw permission error.
- File names: export and screenshot names are sanitised (`sanitize_file_name`): characters Windows forbids (`<>:"/\|?*`) become `_`, trailing dots/spaces are dropped and reserved names such as `CON` or `NUL` are prefixed.
- ffmpeg: after an install through winget or Chocolatey the new PATH only reaches newly started programs, so `resolve_ffmpeg_path` also looks in `%LOCALAPPDATA%\Microsoft\WinGet\Links`, the winget package folder and `%ChocolateyInstall%\bin`.
- Paths from `canonicalize` lose their `\\?\` prefix before being shown or stored (`display_path`).
- CI runs the Rust tests and the converter tests on Windows, plus `windows-end-to-end`: the ignored end-to-end test, which finds Python, creates the managed venv, installs dependencies, generates a mock dataset, converts and loads it.

Troubleshooting

- "Python not found": install Python 3.9+ from python.org (on Windows tick **Add python.exe to PATH**; the Microsoft Store version does not work), then restart the app. On Windows the `py` launcher is also found.

- Converter setup fails: the first conversion needs internet. Behind a proxy or firewall, install the packages yourself (`python -m pip install -r python/requirements.txt`) and start the app from that environment; it prefers an active venv or conda env. The Conversion log has the pip output.
- Windows, "path too long" during setup: enable long paths (Settings > System > For developers > *Enable long paths*, or the `LongPathsEnabled` policy) and convert again.
- Windows, "Could not replace ... it is open in another program": the `.stviz` is open in another stviz-animate window. Close it there and convert again.
- Windows, conversion or export blocked or very slow: some antivirus tools scan every file the converter writes. Excluding `%APPDATA%\stviz-animate` helps.
- Video export uses the OpenCV fallback: install ffmpeg for faster, smaller exports (`winget install Gyan.FFmpeg`, `brew install ffmpeg`, `sudo apt install ffmpeg`, or **Faster video encoding** in the Output panel). winget and Chocolatey installs are found without a restart.
- Workspace cannot find its dataset: put the `.stviz` next to the workspace file.
- Export artifacts: verify PNG frames to isolate encoding artifacts.
- WSL GPU: performance may be limited vs native Windows/Linux.
- Start over: close the app and delete the user-data folder. Converted files, logs, settings and the managed Python environment are all inside it.

Extension points

Common areas to modify:

- Add new export options: `start_export_loop` and `finish_export_loop` in `src/app/export.rs`; anything slow that runs after the last frame belongs in `run_encode` in `src/app/export_job.rs`, not on the UI thread.
- Add new color modes: `ColorMode` enum in `src/app/mod.rs` and `colors_for_key` in `src/app/keyframes.rs`.
- Modify shader rendering: `assets/pointcloud.wgsl`.
- Adjust file format: `python/export_stviz.py` and `src/data.rs`.

1.0 smoke-test checklist

Run before tagging:

1. Load a small `.stviz` and verify interaction/color/filtering.
2. Load a large `.stviz` and verify performance and stability.
3. Switch spaces and color modes (categorical/continuous/gene).
4. Start loop export, then cancel; verify only managed temp outputs are removed.
5. Complete loop export with ffmpeg and (optionally) OpenCV fallback.
6. Run one `.h5ad` conversion with dependencies preinstalled.